Actividad Guiada 1 de Algoritmos de Optimizacion

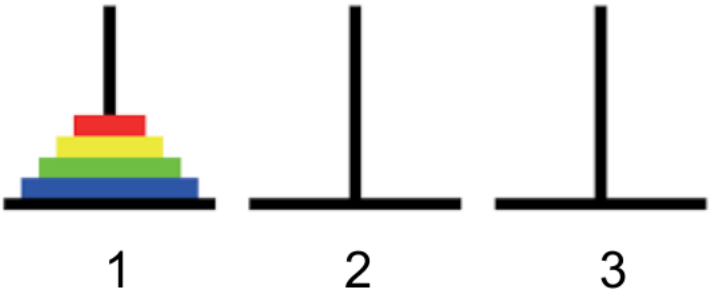
Nombre: Ventura Esteban Guerrero

https://colab.research.google.com/github/venturaesteban/03MIAR-Algoritmos-de-Optimizaci-n/blob/main/notebooks/AG1/Ventura_Esteban_Guerrero_AG1.ipynb

https://github.com/venturaesteban/03MIAR-Algoritmos-de-Optimizaci-n

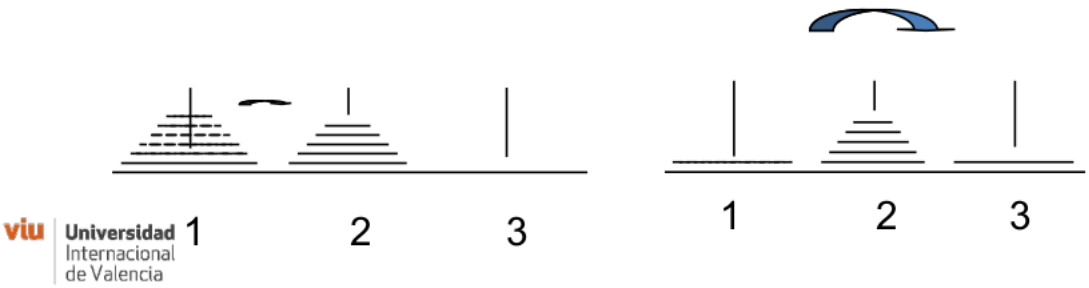
1. Elemento de lista
2. Elemento de lista

Torres de Hanoi - Divide y venceras



Resolver(Total_fichas=4, Desde=1 , Hasta=3) es valido con:

- Resolver(Total_fichas=3, Desde=1, Hasta=2)
- Mover(Desde=1, Hasta=3)
- Resolver(Total_fichas=3, Desde=2, Hasta=3)




```
#Torres de Hanoi - Divide y venceras
##############################################################################


#################################################################
def Torres_Hanoi(N, desde, hasta):
  #N - Nº de fichas
  #desde - torre inicial
  #hasta - torre fina
  if N==1 :
    print("Lleva la ficha desde " + str(desde) + " hasta " + str(hasta))

  else:
    Torres_Hanoi(N-1, desde, 6-desde-hasta)
    print("Lleva la ficha desde " + str(desde) + " hasta " + str(hasta))
    Torres_Hanoi(N-1, 6-desde-hasta,  hasta)
```

```
Torres_Hanoi(5, 1, 3)
################################################################
```

```
Lleva la ficha desde 1 hasta 3
Lleva la ficha desde 1 hasta 2
Lleva la ficha desde 3 hasta 2
Lleva la ficha desde 1 hasta 3
Lleva la ficha desde 2 hasta 1
Lleva la ficha desde 2 hasta 3
Lleva la ficha desde 1 hasta 3
Lleva la ficha desde 1 hasta 2
Lleva la ficha desde 3 hasta 2
Lleva la ficha desde 3 hasta 1
Lleva la ficha desde 2 hasta 1
Lleva la ficha desde 3 hasta 2
Lleva la ficha desde 1 hasta 3
Lleva la ficha desde 1 hasta 2
Lleva la ficha desde 3 hasta 2
Lleva la ficha desde 1 hasta 3
Lleva la ficha desde 2 hasta 1
Lleva la ficha desde 2 hasta 3
Lleva la ficha desde 1 hasta 3
Lleva la ficha desde 2 hasta 1
Lleva la ficha desde 3 hasta 2
Lleva la ficha desde 3 hasta 1
Lleva la ficha desde 2 hasta 1
Lleva la ficha desde 2 hasta 3
Lleva la ficha desde 1 hasta 3
Lleva la ficha desde 1 hasta 2
Lleva la ficha desde 3 hasta 2
Lleva la ficha desde 1 hasta 3
Lleva la ficha desde 2 hasta 1
Lleva la ficha desde 2 hasta 3
Lleva la ficha desde 1 hasta 3
```

Cambio de monedas - Técnica voraz

↳ 2 celdas ocultas

N Reinas - Vuelta Atrás(Backtracking)

```
#N Reinas - Vuelta Atrás()
##################################################################################

#Verifica que en la solución parcial no hay amenzas entre reinas
################################################################
def es_prometedora(SOLUCION,etapa):
################################################################
  #print(SOLUCION)
  #Si la solución tiene dos valores iguales no es valida => Dos reinas en la misma fila
  for i in range(etapa+1):
    #print("El valor " + str(SOLUCION[i]) + " está " +  str(SOLUCION.count(SOLUCION[i])) + " veces")
    if SOLUCION.count(SOLUCION[i]) > 1:
      return False

    #Verifica las diagonales
    for j in range(i+1, etapa +1 ):
      #print("Comprobando diagonal de " + str(i) + " y " + str(j))
      if abs(i-j) == abs(SOLUCION[i]-SOLUCION[j]) : return False
  return True

#Traduce la solución al tablero
```

```
#################################################################
def escribe_solucion(S):
#################################################################
  n = len(S)
  for x in range(n):
    print("")
    for i in range(n):
      if S[i] == x+1:
        print(" X " , end="")
      else:
        print(" - ", end="")



#Proceso principal de N-Reinas
#################################################################
def reinas(N, solucion=[],etapa=0):
#################################################################
### ....
  if len(solucion) == 0:         # [0,0,0...]
    solucion = [0 for i in range(N) ]

  for i in range(1, N+1):
    solucion[etapa] = i
    if es_prometedora(solucion, etapa):
      if etapa == N-1:
        print(solucion)
      else:
        reinas(N, solucion, etapa+1)
    else:
      None

  solucion[etapa] = 0

reinas(6,solucion=[],etapa=0)
```

```
[2, 4, 6, 1, 3, 5]
[3, 6, 2, 5, 1, 4]
[4, 1, 5, 2, 6, 3]
[5, 3, 1, 6, 4, 2]
```

```
escribe_solucion([2, 4, 6, 1, 3, 5])
print("\n")
escribe_solucion([3, 6, 2, 5, 1, 4])
print("\n")
escribe_solucion([4, 1, 5, 2, 6, 3])
print("\n")
escribe_solucion([5, 3, 1, 6, 4, 2])
```

```
 -  -  -  X  -  -
 X  -  -  -  -  -
 -  -  -  -  X  -
 -  X  -  -  -  -
 -  -  -  -  -  X
 -  -  X  -  -  -


 -  -  -  -  X  -
 -  -  X  -  -  -
 X  -  -  -  -  -
 -  -  -  -  -  X
 -  -  -  X  -  -
 -  X  -  -  -  -


 -  X  -  -  -  -
 -  -  -  X  -  -
 -  -  -  -  -  X
 X  -  -  -  -  -
 -  -  X  -  -  -
 -  -  -  -  X  -


 -  -  X  -  -  -
 -  -  -  -  -  X
 -  X  -  -  -  -
 -  -  -  -  X  -
 X  -  -  -  -  -
 -  -  -  X  -  -
```

Encuentra los puntos más cercanos de una lista dada

```
# recorre la lista comparando cada posición con sus sucesoras para encontrar la menor distancia
##########################################################################################
def encuentra_puntos_cercanos(lista):
  if len(lista) <= 2:
    return lista

  # Almacenamos las posiciones 0 y 1 como inciales
  p1 = 0
  p2 = 1
  mejor_dist = abs(lista[p1] - lista[p2]) #Calculamos la distancia de esas posiciones, la cual a priori es la mejor

  for i in range(len(lista)-1):
    for j in range(i+1, len(lista)): # Recorremos posiciones posteriores a i para comparar, las anteriores no tiene sentido
      dist = abs(lista[i] - lista[j])
      if dist < mejor_dist:
        mejor_dist = dist #Si la distancia nueva es menor nos la guardamos como mejor distancia
        p1,p2 = i,j #almacenamos las posiciones de la mejor distancia

  puntos = [lista[p1],lista[p2]] # Creamos un array de 2 elementos con los puntos más cercanos

  return puntos


encuentra_puntos_cercanos([99,55,42,84,684,1351,8718,1,54])
```

```
[55, 54]
```

Para sacar el orden de complejidad de la función, vamos a contar la operaciones elementales que se realizan por línea:

Línea	Operaciones elementales	Coste por ejecución	Nº Veces	Total
`if len(lista) <= 2`	llamada + condición	2	1	2
`return lista`	retorno	1	1	1
`p1 = 0`	asignación	1	1	1
`p2 = 1`	asignación	1	1	1
`mejor_dist = abs(lista[p1] - lista[p2])`	2 accesos + 1 resta + 1 llamada + 1 asignación	5	1	5
`Control for i`	condición + avance	2	n	$2n$
`Control for j`	condición + avance	2	m	$2m$
`dist = abs(lista[i] - lista[j])`	2 accesos + 1 resta + 1 llamada + 1 asignación	5	m	$5m$
`if dist < mejor_dist:`	condición	1	m	m
`dentro del if: mejor_dist=dist; p1=i; p2=j`	3 asignaciones	3	u*	$3u$
`puntos = [lista[p1], lista[p2]]`	2 accesos + 1 asignación	3	1	3
`return puntos`	retorno	1	1	1

$*u$ es el número de veces que se cumple la condición dist < mejor_dist, es decir, cuántas veces entra al if y actualiza la distancia.

Si realizamos la suma obtenemos que:

`T(n)=(2+1+1+1+5+3+1)+2n+(2m+5m+m)+3u` $\Rightarrow$ $T(n)=14+2n+8m+3u$

Siendo m el número de iteración del bucle interno y por tanto una combinación donde comparamos todas las parejas distintas de n elementos $\binom{n}{2}$, podemos afirmar que $m = \frac{n(n-1)}{2}$

Al ser m el termino dominante en la ecuación $8m \Rightarrow 8\frac{n(n-1)}{2} \Rightarrow 4n(n-1) \Rightarrow 4n^2 - 4 \Rightarrow O(n^2)$

```
# Ordena una lista dada por parametros usando el algoritmo Merge Sort
##########################################
def ordena_lista(lista):
  if len(lista) > 1:
    mid = len(lista)//2 #Localizamos el punto medio
    izquierda = lista[:mid]
    derecha = lista[mid:]

    # Llamamos con recursividad para dividir en dos partes los listados
    ordena_lista(izquierda)
    ordena_lista(derecha)

    i = j = k = 0

    # vamos mirando que cual de las dos partes es menor y nos la guardamos
    while i < len(izquierda) and j < len(derecha):
        if izquierda[i] < derecha[j]:
            lista[k] = izquierda[i]
            i += 1
        else:
            lista[k] = derecha[j]
            j += 1
        k += 1

    # Copiamos los elementos de la izquierda tal cual esten
    while i < len(izquierda):
        lista[k] = izquierda[i]
        i += 1
        k += 1

    # Lo mismo con la derecha ya que están ya ordenador
    while j < len(derecha):
        lista[k] = derecha[j]
        j += 1
        k += 1


# Devuelve un par de valores más cercano (en valor absoluto) de una lista que ya esté ordenada.
# La funcion usa el algoritmo de divide y venceras
##########################################
def mas_cercanos(lista,inicio,fin):
  n_puntos = fin - inicio

  # Caso base si no existe un par posible
  if n_puntos < 2:
      return None

  # Caso base: único par posible
  if n_puntos == 2:
      p1, p2 = lista[inicio], lista[inicio + 1]
      return (p1, p2)

  mid = (inicio + fin) // 2 # Encontramos el medio

  # Usamos recursividad para encontrar el mejor par en cada mitad
  valores_izq = mas_cercanos(lista, inicio, mid)
  valores_der = mas_cercanos(lista, mid, fin)

  # Encontramos la frontera, es decir, el par que cruza el corte.
  p1, p2 = lista[mid - 1], lista[mid]
  valores_frontera = (p1, p2)

  # Calculamos las distancias
  distancia_izq = abs(valores_izq[1] - valores_izq[0]) if valores_izq is not None else float("inf")
  distancia_der = abs(valores_der[1] - valores_der[0]) if valores_der is not None else float("inf")
  distancia_frontera = abs(valores_frontera[1] - valores_frontera[0])

  # Elegimos el par con menor distancia y se devuelve solo ese par
  if distancia_izq <= distancia_der and distancia_izq <= distancia_frontera:
      return valores_izq
  elif distancia_der <= distancia_izq and distancia_der <= distancia_frontera:
      return valores_der
  else:
      return valores_frontera
```

```
# Función que encuentra los puntos mas cercanos de una lista dada por parametro
##########################################
def encuentra_puntos_cercanos(lista):
  n_valores = len(lista)
  if n_valores < 2:
      return lista

  ordena_lista(lista)

  puntos_cercanos = mas_cercanos(lista, 0, n_valores)

  return puntos_cercanos



encuentra_puntos_cercanos([21,10,100022,78,56,3213,999,6,52,14,61,35,800,752,625,526,546])
```

```
(6, 10)
```

En este caso, aplicando el algoritmo de divide y venceras, nuestra función consta de dos fases: primero ordena la lista con Merge Sort, cuyo coste es conocido $O(n \cdot log2(n))$. Después, la función mas_cercanos trabaja sobre la lista ya ordenada y en cada llamada el mismo trabajo, con un número total de llamadas con complejidad n, por lo que su complejidad es $O(n)$.

Sumando los costes, la complejidad total queda en: $T(n) = O(n) + O(n \cdot log2(n)) = O(n \cdot log2(n))$